Content-Based Image Retrieval with Parallel NMS-Sift and Multithreading

...

By Musa Waghu

Introduction

- Since the dot com era, there has been a significant increase in the amount of data available on the internet. In 2022 alone, it has been reported that 97 zettabytes of data was created. [1]
- Images are a huge chunk of that data, so it is necessary that we have efficient technologies that can process that amount of data.
- Content-Based Image Retrieval is a type of image retrieval that is different from regular Text-Based Image Retrieval.
- In CBIR, a user specifies a query image and gets the images in the database similar to the query image. To find the most similar images, CBIR compares the content of the input image to the database images. [2]
- CBIR compares visual features such as shapes, colors, texture and spatial information and measures the similarity between the query image with the images in the database with respect to those features. [2]

CBIR vs TBIR

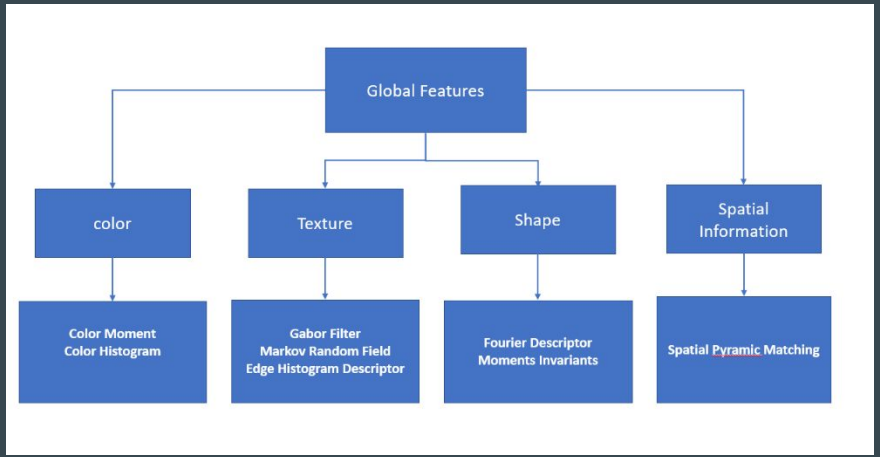
- The differences between CBIR and TBIR is that in Text-Based Image Retrieval, experts manually annotate images with geolocation, keywords, tags, labels, or short descriptions. Users can use keywords, annotations, or descriptions to retrieve similar images that exist in the database.
- Some disadvantages of TBIR are:
  - it takes a considerable amount of time to annotate images manually
  - the tags may be unreliable due to the dependency on people's perceptions and interpretations that can vary widely across groups of individuals
- CBIR compares the visual contents of images directly, which reduces human error and labor. [6]

Disadvantages of CBIR

- The Semantic Gap:The semantic gap refers to the difference between the raw data obtained from visual information, i.e images, and the meaningful understanding of that data by a person in a specific context.
- Scalability: CBIR systems can be computationally intensive, especially when dealing with large image databases. Searching for similarities in high-dimensional feature spaces can be slow and resource-intensive, making real-time or near-real-time retrieval challenging for large-scale image collections.

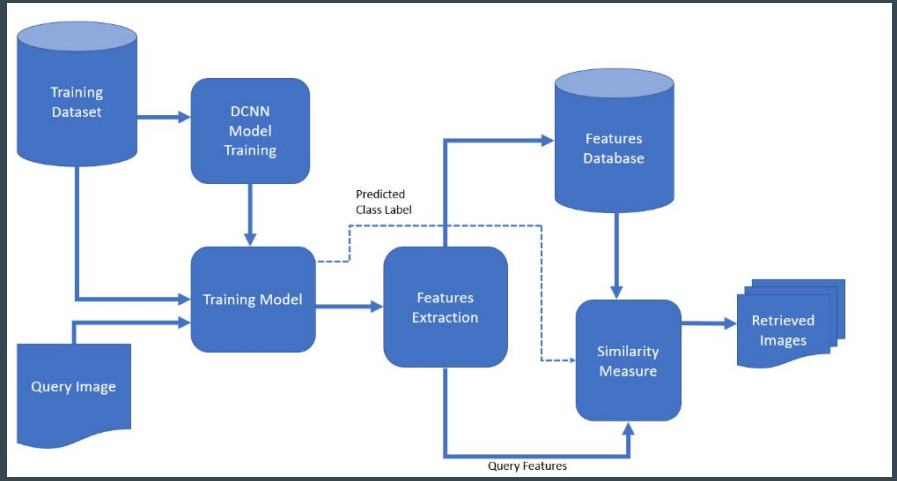
Feature Extraction

- Feature extraction refers to the process of transforming raw data into numerical features that can be processed while preserving the information in the original data set.
- There are two main categories of visual features: global and local. [3]
- Global features are those that describe an entire image. They contain information on the entire image. For example, several attributes characterize color spaces, shapes and textures.



Feature Extraction

- In this project, we will be focusing on local features.
- Local features describe visual patterns or structures identifiable in small groups of pixels. For example, edges, points, and various image patches.
- One of the most used local feature algorithm is SIFT which stands for Scale-Invariant Feature Transform.

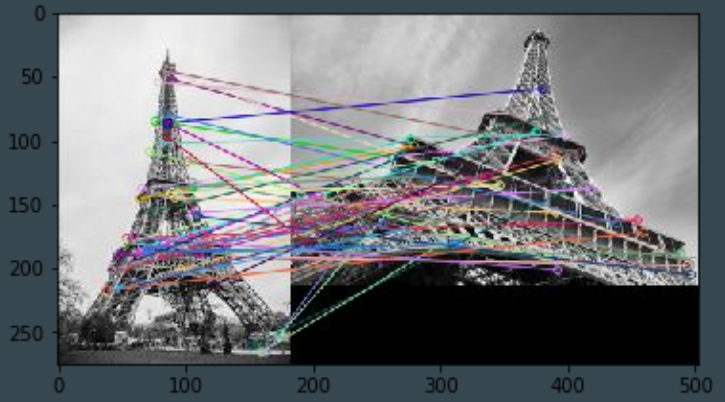


Scale Invariant Feature Transform (SIFT) Algorithm

- SIFT is a CBIR algorithm for extracting features from an image. It consists of a descriptor and a detector for key points. It is invariant to rotation and scale. [4]
- Advantages
  - Locality: features are local, so robust to occlusion and clutter
  - Distinctiveness: individual features can be matched to a large database of objects
  - Efficiency: close to real-time performance
- Disadvantages
  - Mathematically complicated and computationally heavy.
  - SIFT is based on the Histogram of Gradients. That is, the gradients of each Pixel in the patch need to be computed and these computations cost time.
  - High memory usage

Methodology of SIFT

- There are 4 steps to SIFT algorithm
    - Scale-space peak selection: The first stage of computation searches over all scales and image locations.
    - Keypoint Localization: At each candidate location, a detailed model is fit to determine location and scale.
    - Orientation Assignment: One or more orientations are assigned to each keypoint location based on local image gradient directions.
    - Keypoint descriptor: The local image gradients are measured at the selected scale in the region around each keypoint.



Methodology of SIFT

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

Blurred image

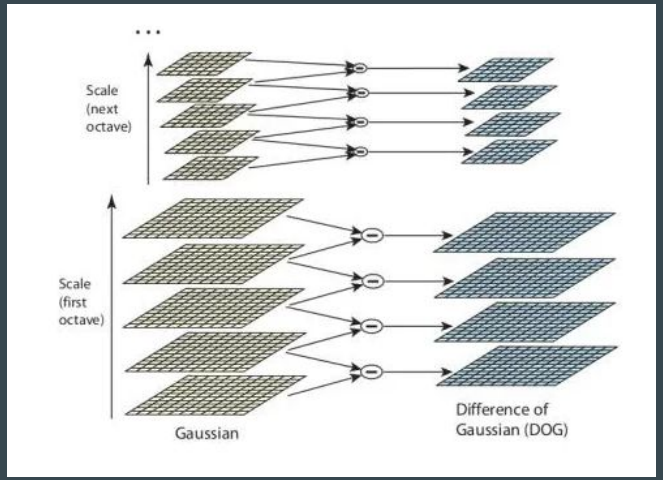
1. **Scale-space peak selection**
    a. The scale space of an image is a function L(x,y,σ) that is produced from the convolution of a variable-scale Gaussian, G(x, y, σ), with an input image, I(x, y)
    b. Scale-space is separated into octaves and the number of octaves and scale depends on the size of the original image. So we generate several octaves of the original image. Each octave's image size is half the previous one.
        i. **Blurring:**
            1. Gaussian blur has a particular expression or "operator" that is applied to each pixel. What results is the blurred image.
            2. G is the Gaussian Blur operator and I is an image. While x,y are the location coordinates and σ is the "scale" parameter. Think of it as the amount of blur. Greater the value, greater the blur.

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$$

Gaussian Blur operator

Methodology of SIFT

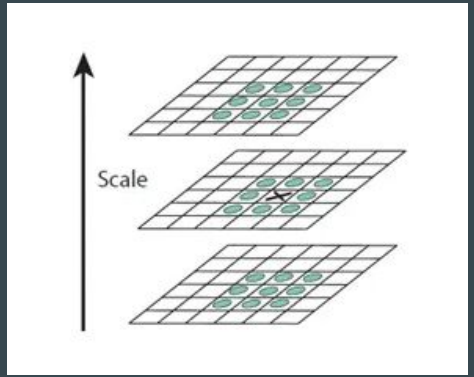
- **Difference of Gaussian Kernel(DoG):**
  - Using the blurred images from before, we generate the (DoG). These DoG images are great for finding out interesting key points in the image.
  - The difference of Gaussian is obtained as the difference of Gaussian blurring of an image with two different σ, let it be σ and kσ, as shown in the image to the right.
  - This process is done for different octaves of the image in the Gaussian Pyramid.



$$D(x, y, \sigma) = (G(x, y, k\sigma) - G(x, y, \sigma)) * I(x, y)$$
$$= L(x, y, k\sigma) - L(x, y, \sigma).$$

Methodology of SIFT

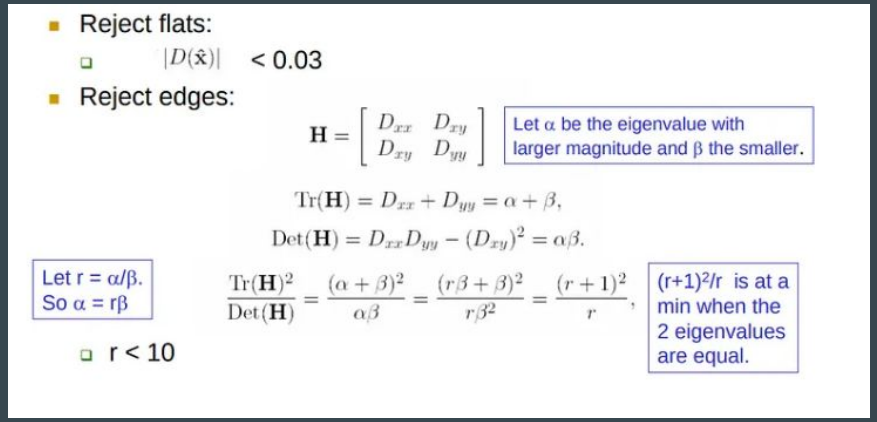
- **Finding key points:**
  - Using the DoG, we calculate the Laplacian of Gaussian approximations that are scale invariant.
  - One pixel in an image is compared with its 8 neighbors as well as 9 pixels in the next scale and 9 pixels in previous scales. This way, a total of 26 checks are made.
  - If it is a local extrema, it is a potential keypoint.



Methodology of SIFT

2. **Key point localization** :

- A lot of key points either lie on an edge or don't have enough contrast, so they have to be eliminated.
- Use Taylor Series to get a more accurate location of the extrema, and if the intensity is below a certain threshold, it is discarded.
- Use a 2x2 Hessian matrix to compute the principal curvature.
    - Hessian matrix is a matrix of second order partial derivatives of the intensity values



Methodology of SIFT

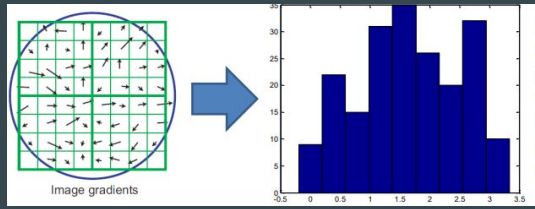
3. Orientation Assignment

- The next thing is to assign an orientation to each keypoint to make it rotation invariance.
- A neighborhood is taken around the keypoint location depending on the scale, and the gradient magnitude and direction is calculated in that region.
- m(x,y) = represents the magnitude of the change in pixel intensity at a given point (x,y) in an image
- θ(x,y) = represents the angle of the gradient at a given point (x,y) in an image

$$m(x,y) = \sqrt{(L(x+1,y) - L(x-1,y))^2 + (L(x,y+1) - L(x,y-1))^2}$$

$$\theta(x,y) = \tan^{-1}((L(x,y+1) - L(x,y-1))/(L(x+1,y) - L(x-1,y)))$$

For each image sample, L(x, y), at this scale, the gradient magnitude, m(x, y), and orientation, θ(x, y), is precomputed using pixel differences:



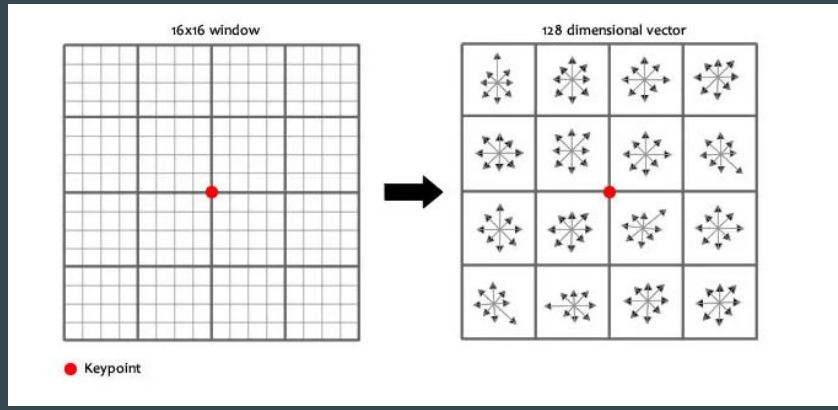
Histogram of gradient orientation – the bin-counts are weighted by gradient magnitudes and a Gaussian weighting function. Usually, 36 bins are chosen for the orientation.

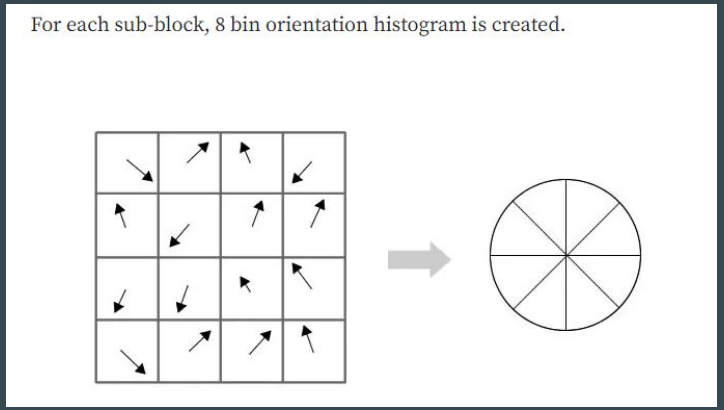
Peaks in the histogram corresponds to dominant direction of local gradients.
The highest peak in the histogram is taken and any peak above 80% of it is also considered to calculate the orientation.

Methodology of SIFT

4. Keypoint descriptor

- At this point, each keypoint has a location, scale, orientation.
- Next is to compute a descriptor for the local image region about each keypoint that is highly distinctive and invariant as possible to variations such as changes in viewpoint and illumination.
- To do this, a 16x16 window around the keypoint is taken. It is divided into 16 sub-blocks of 4x4 size.





NMS Introduction

- We can further improve the implementation the SIFT algorithm.
- One technique that can help is NMS (Non-Maximum Suppression).

NMS

- NMS eliminates redundant or overlapping keypoints.
- It works by comparing the confidence scores of the proposed bounding boxes and eliminating the ones that overlap significantly with a higher-scoring bounding box. [5]
- The overlapping regions between the bounding boxes are typically calculated using the Intersection over Union (IoU) metric.
- If the IoU value between two bounding boxes exceeds a predefined threshold value, the bounding box with the lower confidence score is discarded.

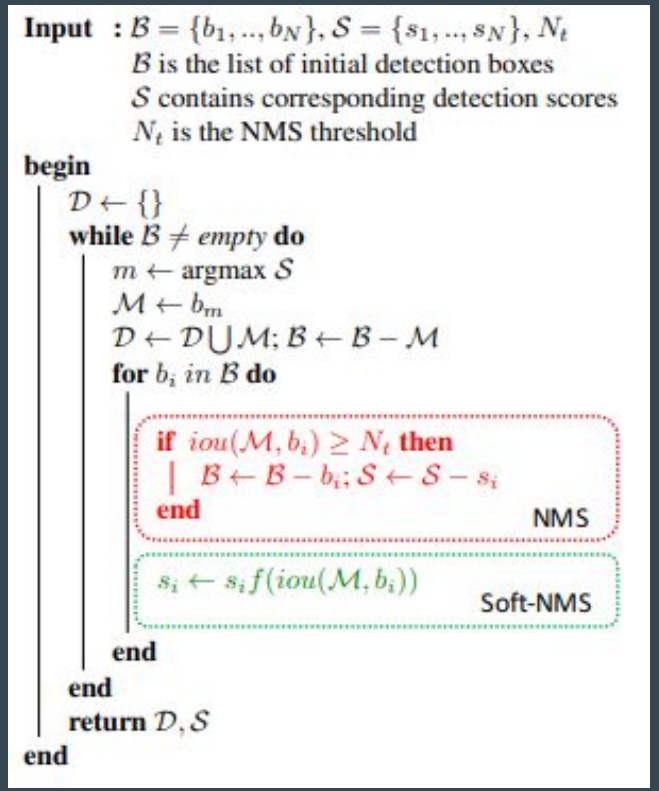


Credit: learnopencv.com



NMS Bounding Box

- Non-maximum suppression starts with a list of detection boxes B with scores S. After selecting the detection with the maximum score M, it removes it from the set B and appends it to the set of final detections D.
- It also removes any box which has an overlap greater than a threshold Nt with M in the set B. This process is repeated for remaining boxes B.



Research Goal

- The goal of this research is to further improve the existing NMS-SIFT algorithm with the introduction of partial processing and multithreading.
- The primary function of multithreading is to simultaneously run or execute multiple tasks.
- In this research problem, our task is to compute the key points of images.
- With the introduction of multithreading, we can divide an image into multiple smaller images and let a separate thread handle each one of them.

Multithreading

- Advantages of multithreading:
  - Improved performance: Multithreading can help increase the overall performance of an application, especially on systems with multiple processors or cores. It allows multiple tasks to run concurrently, utilizing the available CPU resources more efficiently.
  - Responsiveness: In a single-threaded environment, if a long-running task blocks the main thread, the entire application becomes unresponsive. Multithreading can prevent this issue by running such tasks in separate threads, ensuring the application remains responsive.
- Disadvantages of multithreading:
  - Complexity: Multithreading adds complexity to the program, making it more difficult to design, implement, and debug.
  - Unpredictable: Due to the concurrent nature of multithreading, the behavior of the program can be hard to predict and reproduce, especially when it comes to debugging.

Methodology

- In our research, we will divide an image into 16 quadrants rather than the original 4 quadrants.
- Each quadrant is processed independently by a separate thread
- Multiple threads process simultaneously using this method
- This method improves the processing time significantly.

Hypotheses

- The 16 quadrant approach is hypothesized to be more efficient and faster in terms of computation than the 4 quadrant approach.

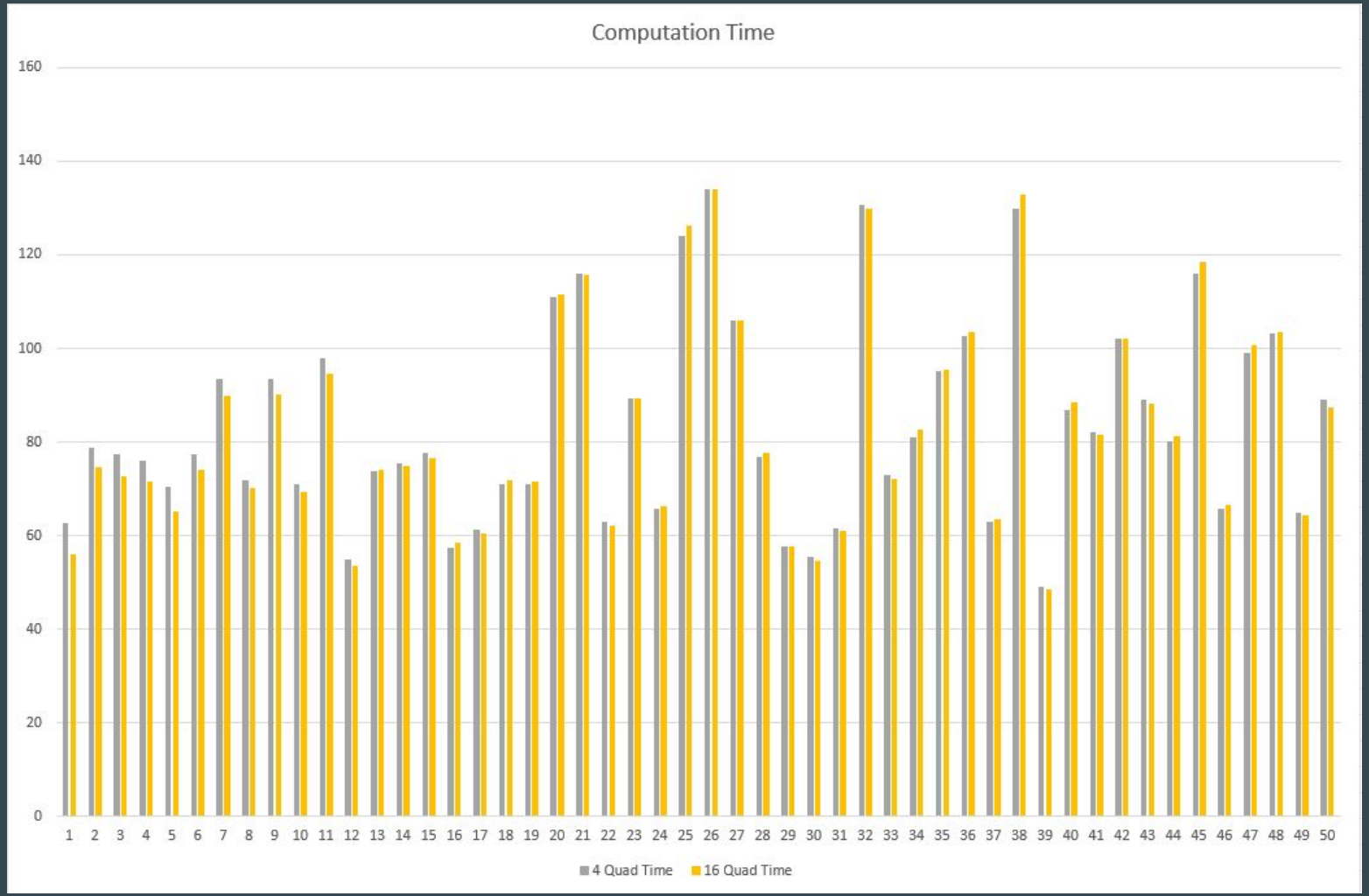
Experimental Setup

- Dataset:
  - 102 Category Flower Dataset by Maria-Elena Nilsback and Andrew Zisserman (Nilsback and Zisserman 2008)
- System Configuration
  - System used: Custom-built PC
  - System CPU: AMD Ryzen 7 3700x (8-core 16-thread)
  - System RAM: 32GB
  - System Memory: 500GB SSD

Results

- Computation time between the two algorithms has been presented in a bar graph.
- The 16 quadrant version outperformed the 4 quadrants in some images, not all.
- The results are unpredictable.
- The difference between the two algorithms is not too significant.
- Furthermore, the more quadrants you have, the less keypoints are generated.

Image Size	Image	4 Quadrants Time	16 Quadrants Time	4 Quadrants Keypoints	16 Quadrants Keypoints	Time Diff	Key Diff
48	1	62.7	56.02338433	808	753	6.6766157	55
56	2	78.8	74.48856878	1175	1119	4.3114312	56
57	3	77.5	72.5443275	1110	1056	4.9556725	54
56	4	76	71.59146166	1104	1052	4.4085383	52
52	5	70.5	65.17818403	1074	988	5.321816	86
44	6	77.5	74.05451322	1170	1106	3.4454868	64
51	7	93.4	89.77150464	1420	1349	3.6284954	71
52	8	71.7	70.03803921	1043	983	1.6619608	60
72	9	93.5	90.14117813	1471	1398	3.3588219	73
48	10	71	69.41249275	1015	980	1.5875072	35
72	11	97.9	94.5204134	1462	1379	3.3795866	83
43	12	54.9	53.55131912	820	766	1.3486809	54
57	13	73.8	74.10616493	1058	1015	-0.306165	43
45	14	75.5	74.90227628	1111	1048	0.5977237	63
66	15	77.6	76.5586648	1090	1026	1.0413352	64
46	16	57.3	58.37470412	760	742	-1.074704	18
51	17	61.2	60.41243815	920	871	0.7875618	49
68	18	71	71.92305279	973	938	-0.923053	35
61	19	70.9	71.51179147	1057	1005	-0.611791	52
63	20	111	111.5247002	1653	1590	-0.5247	63
67	21	116.1	115.5933502	1852	1775	0.5066498	77
52	22	62.8	62.09921288	988	941	0.7007871	47
62	23	89.4	89.3793211	1362	1281	0.0206789	81
48	24	65.8	66.21137691	896	854	-0.411377	42
84	25	123.9	126.3009245	2002	1936	-2.400925	66
64	26	133.9	134.0219083	2075	1997	-0.121908	78
60	27	106	105.9055245	1698	1615	0.0944755	83
67	28	76.8	77.63994122	1151	1092	-0.839941	59
48	29	57.6	57.57737684	777	737	0.0226232	40
46	30	55.3	54.52748132	718	668	0.7725187	50
49	31	61.6	61.08565879	836	782	0.5143412	54
68	32	130.8	129.9017539	2084	1993	0.8982461	91
55	33	72.8	72.20407677	1101	1034	0.5959232	67
53	34	80.9	82.64778328	1128	1084	-1.747783	44
63	35	95.1	95.45705795	1476	1414	-0.357058	62
60	36	102.7	103.5565772	1551	1486	-0.856577	65


Computation Time
160
140
120
100
80
60
40
20
0
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
4 Quad Time
16 Quad Time

Future Direction

- **Vertical or Horizontal Scaling:**
  - One way we can further reduce the computation time is to upgrade the CPU. There are many high performance CPU's, whose thread count can even go up 192 threads. Or we can introduce GPU's to reduce the stress on the CPU. Or we can consider distributing the workload across many machines.
- **Consider smaller quadrants:**
  - Another way could be to consider smaller quadrants, like 32 or even 64. Only problem I see with this is if it is even possible to divide an image that much. Will have to look into their pixels and size.

Bibliography

- [1] “Data Growth Worldwide 2010-2025.” Statista, https://www.statista.com/statistics/871513/worldwide-data-created/. Accessed 25 Nov. 2023.
- [2] W. by: baeldung, “What is content-based image retrieval?,” Baeldung on Computer Science, https://www.baeldung.com/cs/cbir-tbir#:~:text=Content%2DBased%20Image%20Retrieval%20(CBIR)%20is%20a%20way%20of,image%20to%20the%20database%20images (accessed Nov. 20, 2023).
- [3]Kumar, Gaurav, and Pradeep Kumar Bhatia. “A Detailed Review of Feature Extraction in Image Processing Systems.” 2014 Fourth International Conference on Advanced Computing & Communication Technologies, 2014, pp. 5–12. IEEE Xplore, https://doi.org/10.1109/ACCT.2014.74.

Bibliography

- [4] Lowe, D. G. “Object Recognition from Local Scale-Invariant Features.” Proceedings of the Seventh IEEE International Conference on Computer Vision, vol. 2, 1999, pp. 1150–57 vol.2. IEEE Xplore, https://doi.org/10.1109/ICCV.1999.790410.
- [5] Hosang, Jan, et al. “Learning Non-Maximum Suppression.” 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 6469–77. IEEE Xplore, https://doi.org/10.1109/CVPR.2017.685.
- [6] Singh, Mona, et al. “A Comparative Study of Feature Extraction Techniques and Similarity Measures for Image Retrieval.” 2022 International Conference on Futuristic Technologies (INCOFT), IEEE, 2022, pp. 1–7. DOI.org (Crossref), https://doi.org/10.1109/INCOFT55651.2022.10094430.

Acknowledgment

- Code for this project was initially created by Sairaj Jadhav and Dr. John Korah